OS Assignment Unit 4

NAME: Catherine D

SRN: PES1UG24CS120

Sec: B

1A)

```
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ ./text_objects
-bash: ./text_objects: No such file or directory
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$
```

1B)

```
All Phase 1 tests passed.
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$
```

2A)

```
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$
```

2B)

```
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/42/ca534e7eec255f41fb7224e4f4bfd5d1df35fc71bf0f21aeb5f418a7e3ce0d
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d
337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$
```

3A)

```
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ echo "hello" >> file1.txt
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ echo "world" >> file2.txt
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ ./pes add file1.txt file2.txt
Adding file: file1.txt
Adding file: file2.txt
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

3B)

```
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ cat .pes/index
100755 8a633eb173fdd6c0b4c8abff118e352e3c9bb91b3ed79944e12c2ec1e5ed45e5 1776656407 27 file1.txt
100755 087c4f7ee21d7b2d3956a15a84ada1d029ea77254461f7f4e57dabec70ee5fec 1776656412 33 file2.txt
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$
```

4A)

```
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ ./pes log
commit 4e00a53232443906c8bedd18a04dcd7d10744702cbe3addcab1fa866c10a4ff1
Author: Catherine <PES1UG24CS120>
Date:   1776657008

    Add farewell

commit 06c6849071899f09cd061f74cb6e8bddd3464cf2a135acf928daae005a5ec3f8
Author: Catherine <PES1UG24CS120>
Date:   1776656968

    Add worldell

commit 91c33bcf4a9d7f0aa9e4b5986bdbea26a39b0643bb29c5aa405152f8bcf93606
Author: Catherine <PES1UG24CS120>
Date:   1776656879

    Initial commit
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$
```

4B)

```
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/06/c6849071899f09cd061f74cb6e8bddd3464cf2a135acf928daae005a5ec3f8
.pes/objects/08/7c4f7ee21d7b2d3956a15a84ada1d029ea77254461f7f4e57dabec70ee5fec
.pes/objects/4e/00a53232443906c8bedd18a04dcd7d10744702cbe3addcab1fa866c10a4ff1
.pes/objects/63/7fcf16623ffe1a27ad89eb7386f2d6ccb825298302e98dfabd87ce75628d59
.pes/objects/82/ef4cebb7afc20a47ffda706a42181db8f873090649fbcc857b30adba6ff1ff
.pes/objects/8a/633eb173fdd6c0b4c8abff118e352e3c9bb91b3ed79944e12c2ec1e5ed45e5
.pes/objects/91/c33bcf4a9d7f0aa9e4b5986bdbea26a39b0643bb29c5aa405152f8bcf93606
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$
```

4C)

```
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ cat .pes/refs/heads/main
4e00a53232443906c8bedd18a04dcd7d10744702cbe3addcab1fa866c10a4ff1
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
cathy@LAPTOP-GMEK8CGP:/mnt/c/Users/Cathy/Desktop/os/orangeprob4/PES1UG24CS120-pes-vcs$
```

5.1) A branch is just a file storing a commit hash, so pes checkout <branch> mainly means that:

- Update .pes/HEAD : point to the new branch
- Read the commit hash from .pes/refs/heads/<branch>
- Load that commit's tree
- Replace the working directory to match that tree

What makes this complex is that we might overwrite user changes, need to delete files that don't exist in the new branch or nested directories make updates more complex

5.2) To detect a dirty working directory, we need to compare the index with the working directory to see if any tracked files have been modified. Then compare the index with the target branch's tree to check if those same files differ in the branch we are switching to. If a file has been modified locally and also differs in the target branch, checkout should refuse to proceed because switching would overwrite unsaved changes. In simple terms, the system is checking whether any local changes would be lost during the checkout.

5.3) In a detached HEAD state, HEAD points directly to a commit instead of a branch. We can still make commits in this state, but they are not associated with any branch, meaning they can easily become unreachable later. If the user switches away, those commits may appear lost. However, they can still be recovered if the commit hash is known, or by creating a new branch that points to that commit. It is similar to working on something without saving it to a proper location.

Q6.1) Garbage collection works by identifying which objects are still reachable and removing the rest. The process begins from all branch heads, then traverses through commits, trees, and blobs, marking each reachable object. After this traversal, any object that has not been marked can be safely deleted. A hash set is typically used to track reachable objects efficiently because it allows fast lookups. In a repository with around 100,000 commits and multiple branches, the system would visit roughly those commits along with their associated trees and blobs, resulting in a few hundred thousand objects being processed overall.

Q6.2) Running garbage collection concurrently with a commit operation is dangerous because it can lead to a race condition. For example, a commit may write new objects to the object store but has not yet updated the branch reference. If garbage collection runs at that moment, it may consider those objects unreachable and delete them. When the commit finally updates the reference, it ends up pointing to objects that no longer exist, causing corruption. Git avoids this issue by not immediately deleting recent objects, using mechanisms like grace periods and reflogs, and typically running garbage collection only when it is safe to do so.